

12.6 Exercises

Reinforcement

- R-12.1 Give a complete justification of Proposition 12.1.
- R-12.2 In the merge-sort tree shown in Figures 12.2 through 12.4, some edges are drawn as arrows. What is the meaning of a downward arrow? How about an upward arrow?
- R-12.3 Show that the running time of the merge-sort algorithm on an n -element sequence is $O(n \log n)$, even when n is not a power of 2.
- R-12.4 Is our array-based implementation of merge-sort given in Section 12.1.2 stable? Explain why or why not.
- R-12.5 Is our linked-list-based implementation of merge-sort (Code Fragment 12.3) stable? Explain why or why not.
- R-12.6 An algorithm that sorts key-value entries by key is said to be *straggling* if any time two entries e_i and e_j have equal keys, but e_i appears before e_j in the input, then the algorithm places e_i after e_j in the output. Describe a change to the merge-sort algorithm in Section 12.1 to make it straggling.
- R-12.7 Suppose we are given two n -element sorted sequences A and B each with distinct elements, but potentially some elements that are in both sequences. Describe an $O(n)$ -time method for computing a sequence representing the union $A \cup B$ (with no duplicates) as a sorted sequence.
- R-12.8 Give pseudocode descriptions for the `retainAll` and `removeAll` methods of the set ADT, assuming we use sorted sequences to implement sets.
- R-12.9 Suppose we modify the deterministic version of the quick-sort algorithm so that, instead of selecting the last element in an n -element sequence as the pivot, we choose the element at index $\lfloor n/2 \rfloor$. What is the running time of this version of quick-sort on a sequence that is already sorted?
- R-12.10 Consider a modification of the deterministic version of the quick-sort algorithm where we choose the element at index $\lfloor n/2 \rfloor$ as our pivot. Describe the kind of sequence that would cause this version of quick-sort to run in $\Omega(n^2)$ time.
- R-12.11 Suppose the method `quickSortInPlace` is executed on a sequence with duplicate elements. Prove that the algorithm still correctly sorts the input sequence. What happens in the partition step when there are elements equal to the pivot? What is the running time of the algorithm if all the input elements are equal?
- R-12.12 Show that the best-case running time of quick-sort on a sequence of size n with distinct elements is $\Omega(n \log n)$.
- R-12.13 If the outermost while loop of our implementation of `quickSortInPlace` (line 9 of Code Fragment 12.6) were changed to use condition `left < right`, instead of condition `left <= right`, there would be a flaw. Explain the flaw and give a specific input sequence on which such an implementation fails.

- R-12.14 If the conditional at line 14 of our quickSortInPlace implementation of Code Fragment 12.6 were changed to use condition $\text{left} < \text{right}$, instead of condition $\text{left} \leq \text{right}$, there would be a flaw. Explain the flaw and give a specific input sequence on which such an implementation fails.
- R-12.15 Following our analysis of randomized quick-sort in Section 12.2.1, show that the probability that a given input element x belongs to more than $2 \log n$ subproblems in size group i is at most $1/n^2$.
- R-12.16 Of the $n!$ possible inputs to a given comparison-based sorting algorithm, what is the absolute maximum number of inputs that could be correctly sorted with just n comparisons?
- R-12.17 Jonathan has a comparison-based sorting algorithm that sorts the first k elements of a sequence of size n in $O(n)$ time. Give a big-Oh characterization of the biggest that k can be.
- R-12.18 Is the bucket-sort algorithm in-place? Why or why not?
- R-12.19 Describe a radix-sort method for lexicographically sorting a sequence S of triplets (k, l, m) , where k, l , and m are integers in the range $[0, N - 1]$, for $N \geq 2$. How could this scheme be extended to sequences of d -tuples $(k_1, k_2, \dots, k_d)$, where each k_i is an integer in the range $[0, N - 1]$?
- R-12.20 Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S with the merge-sort algorithm? What about quick-sort?
- R-12.21 Suppose S is a sequence of n values, each equal to 0 or 1. How long will it take to sort S stably with the bucket-sort algorithm?
- R-12.22 Given a sequence S of n values, each equal to 0 or 1, describe an in-place method for sorting S .
- R-12.23 Give an example input that requires merge-sort and heap-sort to take $O(n \log n)$ time to sort, but insertion-sort runs in $O(n)$ time. What if you reverse this list?
- R-12.24 What is the best algorithm for sorting each of the following: general comparable objects, long character strings, 32-bit integers, double-precision floating-point numbers, and bytes? Justify your answer.
- R-12.25 Show that the worst-case running time of quick-select on an n -element sequence is $\Omega(n^2)$.

Creativity

- C-12.26 Describe and analyze an efficient method for removing all duplicates from a collection A of n elements.
- C-12.27 Augment the PositionalList class (see Section 7.3) to support a method named `sort` that sorts the elements of a list by relinking existing nodes; you are not to create any new nodes. You may use your choice of sorting algorithm.

- C-12.28 Linda claims to have an algorithm that takes an input sequence S and produces an output sequence T that is a sorting of the n elements in S .
- Give an algorithm, `isSorted`, that tests in $O(n)$ time if T is sorted.
 - Explain why the algorithm `isSorted` is not sufficient to prove a particular output T to Linda's algorithm is a sorting of S .
 - Describe what additional information Linda's algorithm could output so that her algorithm's correctness could be established on any given S and T in $O(n)$ time.
- C-12.29 Augment the `PositionalList` class (see Section 7.3) to support a method named `merge` with the following behavior. If A and B are `PositionalList` instances whose elements are sorted, the syntax `A.merge(B)` should merge all elements of B into A so that A remains sorted and B becomes empty. Your implementation must accomplish the merge by relinking existing nodes; you are not to create any new nodes.
- C-12.30 Implement a bottom-up merge-sort for a collection of items by placing each item in its own queue, and then repeatedly merging pairs of queues until all items are sorted within a single queue.
- C-12.31 Modify our in-place quick-sort implementation of Code Fragment 12.6 to be a *randomized* version of the algorithm, as discussed in Section 12.2.1.
- C-12.32 Consider a version of deterministic quick-sort where we pick as our pivot the median of the d last elements in the input sequence of n elements, for a fixed, constant odd number $d \geq 3$. What is the asymptotic worst-case running time of quick-sort in this case?
- C-12.33 Another way to analyze randomized quick-sort is to use a **recurrence equation**. In this case, we let $T(n)$ denote the expected running time of randomized quick-sort, and we observe that, because of the worst-case partitions for good and bad splits, we can write

$$T(n) \leq \frac{1}{2} (T(3n/4) + T(n/4)) + \frac{1}{2} (T(n-1)) + bn,$$

where bn is the time needed to partition a list for a given pivot and concatenate the result sublists after the recursive calls return. Show, by induction, that $T(n)$ is $O(n \log n)$.

- C-12.34 Our high-level description of quick-sort describes partitioning the elements into three sets L , E , and G , having keys less than, equal to, or greater than the pivot, respectively. However, our in-place quick-sort implementation of Code Fragment 12.6 does not gather all elements equal to the pivot into a set E . An alternative strategy for an in-place, three-way partition is as follows. Loop through the elements from left to right maintaining indices a , b , and c and the invariant that elements with index i such that $0 \leq i < a$ are strictly less than the pivot, those with $a \leq i < b$ are equal to the pivot, and those with index $b \leq i < c$ are strictly greater than the pivot; elements with index $c \leq i < n$ are yet unclassified. In each pass of the loop, classify one additional element, performing a constant number of swaps as needed. Implement an in-place quick-sort using this strategy.

- C-12.35 Suppose we are given an n -element sequence S such that each element in S represents a different vote for president, where each vote is given as an integer representing a particular candidate, yet the integers may be arbitrarily large (even if the number of candidates is not). Design an $O(n \log n)$ -time algorithm to see who wins the election S represents, assuming the candidate with the most votes wins.
- C-12.36 Consider the voting problem from Exercise C-12.35, but now suppose that we know the number $k < n$ of candidates running, even though the integer IDs for those candidates can be arbitrarily large. Describe an $O(n \log k)$ -time algorithm for determining who wins the election.
- C-12.37 Consider the voting problem from Exercise C-12.35, but now suppose the integers 1 to k are used to identify $k < n$ candidates. Design an $O(n)$ -time algorithm to determine who wins the election.
- C-12.38 Show that any comparison-based sorting algorithm can be made to be stable without affecting its asymptotic running time.
- C-12.39 Suppose we are given two sequences A and B of n elements, possibly containing duplicates, on which a total order relation is defined. Describe an efficient algorithm for determining if A and B contain the same set of elements. What is the running time of this method?
- C-12.40 Given an array A of n integers in the range $[0, n^2 - 1]$, describe a simple method for sorting A in $O(n)$ time.
- C-12.41 Let $S_1, S_2, \dots, S_k$ be k different sequences whose elements have integer keys in the range $[0, N - 1]$, for some parameter $N \geq 2$. Describe an algorithm that produces k respective sorted sequences in $O(n + N)$ time, where n denotes the sum of the sizes of those sequences.
- C-12.42 Given a sequence S of n elements, on which a total order relation is defined, describe an efficient method for determining whether there are two equal elements in S . What is the running time of your method?
- C-12.43 Let S be a sequence of n elements on which a total order relation is defined. Recall that an ***inversion*** in S is a pair of elements x and y such that x appears before y in S but $x > y$. Describe an algorithm running in $O(n \log n)$ time for determining the ***number*** of inversions in S .
- C-12.44 Let S be a sequence of n integers. Describe a method for printing out all the pairs of inversions in S in $O(n + k)$ time, where k is the number of such inversions.
- C-12.45 Let S be a random permutation of n distinct integers. Argue that the expected running time of insertion-sort on S is $\Omega(n^2)$. (Hint: Note that half of the elements ranked in the top half of a sorted version of S are expected to be in the first half of S .)
- C-12.46 Let A and B be two sequences of n integers each. Given an integer m , describe an $O(n \log n)$ -time algorithm for determining if there is an integer a in A and an integer b in B such that $m = a + b$.

- C-12.47 Given two sets A and B represented as sorted sequences, describe an efficient algorithm for computing $A \oplus B$, which is the set of elements that are in A or B , but not in both.
- C-12.48 Given a set of n integers, describe and analyze a fast method for finding the $\lceil \log n \rceil$ integers closest to the median.
- C-12.49 Bob has a set A of n nuts and a set B of n bolts, such that each nut in A has a unique matching bolt in B . Unfortunately, the nuts in A all look the same, and the bolts in B all look the same as well. The only kind of a comparison that Bob can make is to take a nut-bolt pair (a, b) , such that a is in A and b is in B , and test it to see if the threads of a are larger, smaller, or a perfect match with the threads of b . Describe and analyze an efficient algorithm for Bob to match up all of his nuts and bolts.
- C-12.50 Our quick-select implementation can be made more space-efficient by initially computing only the *counts* for sets L , E , and G , and creating only the new subset that will be needed for recursion. Implement such a version.
- C-12.51 Describe an in-place version of the quick-select algorithm in pseudocode, assuming that you are allowed to modify the order of elements.
- C-12.52 Show how to use a deterministic $O(n)$ -time selection algorithm to sort a sequence of n elements in $O(n \log n)$ -*worst-case* time.
- C-12.53 Given an unsorted sequence S of n comparable elements, and an integer k , give an $O(n \log k)$ -expected-time algorithm for finding the $O(k)$ elements that have rank $\lceil n/k \rceil$, $2\lceil n/k \rceil$, $3\lceil n/k \rceil$, and so on.
- C-12.54 Space aliens have given us a method, `alienSplit`, that can take a sequence S of n integers and partition S in $O(n)$ time into sequences $S_1, S_2, \dots, S_k$ of size at most $\lceil n/k \rceil$ each, such that the elements in S_i are less than or equal to every element in S_{i+1} , for $i = 1, 2, \dots, k-1$, for a fixed number, $k < n$. Show how to use `alienSplit` to sort S in $O(n \log n / \log k)$ time.
- C-12.55 Show that randomized quick-sort runs in $O(n \log n)$ time with probability at least $1 - 1/n$, that is, with **high probability**, by answering the following:
- For each input element x , define $C_{i,j}(x)$ to be a 0/1 random variable that is 1 if and only if element x is in $j+1$ subproblems that have size s such that $(3/4)^{i+1}n < s \leq (3/4)^i n$. Argue why we need not define $C_{i,j}$ for $j > n$.
 - Let $X_{i,j}$ be an independent 0/1 random variable that is 1 with probability $1/2^j$, and let $L = \lceil \log_{4/3} n \rceil$. Argue that $\sum_{i=0}^{L-1} \sum_{j=0}^n C_{i,j}(x) \leq \sum_{i=0}^{L-1} \sum_{j=0}^n X_{i,j}$.
 - Show that the expected value of $\sum_{i=0}^{L-1} \sum_{j=0}^n X_{i,j}$ is $(2 - 1/2^n)L$.
 - Show that the probability that $\sum_{i=0}^L \sum_{j=0}^n X_{i,j} > 4L$ is at most $1/n^2$, using the **Chernoff bound** that states that if X is the sum of a finite number of independent 0/1 random variables, having expected value $\mu > 0$, then $\Pr(X > 2\mu) < (4/e)^{-\mu}$, where $e = 2.71828128\dots$.
 - Argue that randomized quick-sort runs in $O(n \log n)$ time with probability at least $1 - 1/n$.

C-12.56 We can make the quick-select algorithm deterministic, by choosing the pivot of an n -element sequence as follows:

Partition the set S into $\lceil n/5 \rceil$ groups of size 5 each (except possibly for one group). Sort each little set and identify the median element in this set. From this set of $\lceil n/5 \rceil$ “baby” medians, apply the selection algorithm recursively to find the median of the baby medians. Use this element as the pivot and proceed as in the quick-select algorithm.

Show that this deterministic quick-select algorithm runs in $O(n)$ time by answering the following questions (please ignore floor and ceiling functions if that simplifies the mathematics, for the asymptotics are the same either way):

- a. How many baby medians are less than or equal to the chosen pivot? How many are greater than or equal to the pivot?
- b. For each baby median less than or equal to the pivot, how many other elements are less than or equal to the pivot? Is the same true for those greater than or equal to the pivot?
- c. Argue why the method for finding the deterministic pivot and using it to partition S takes $O(n)$ time.
- d. Based on these estimates, write a recurrence equation to bound the worst-case running time $t(n)$ for this selection algorithm (note that in the worst case there are two recursive calls—one to find the median of the baby medians and one to recur on the larger of L and G).
- e. Using this recurrence equation, show by induction that $t(n)$ is $O(n)$.

C-12.57 Suppose we are interested in dynamically maintaining a set S of integers, which is initially empty, while supporting the following two operations:

add(v): Adds value v to set S .

median(): Returns the current median value of the set. For a set with even cardinality, we define the median as the average of the two most central values.

We will store each element of the set in one of two priority queues: a min-oriented priority queue, Q^+ , of all elements greater than or equal to the current median value, and a max-oriented priority queue, Q^- , of all elements less than the current median value.

- a. Explain how to perform the operation **median()** in $O(1)$ time given such a representation.
- b. Explain how to perform the operation $S.add(k)$ in $O(\log n)$ time, where n is the current cardinality of the set, while maintaining such a representation.

C-12.58 As a generalization of the previous problem, revisit Exercise C-11.45, which involves performing general selection queries on a dynamic set of values.

Projects

- P-12.59 Implement a nonrecursive, in-place version of the quick-sort algorithm, as described at the end of Section 12.2.2.
- P-12.60 Experimentally compare the performance of in-place quick-sort and a version of quick-sort that is not in-place.
- P-12.61 Perform a series of benchmarking tests on a version of merge-sort and quick-sort to determine which one is faster. Your tests should include sequences that are “random” as well as “almost” sorted.
- P-12.62 Implement deterministic and randomized versions of the quick-sort algorithm and perform a series of benchmarking tests to see which one is faster. Your tests should include sequences that are very “random” looking as well as ones that are “almost” sorted.
- P-12.63 Implement an in-place version of insertion-sort and an in-place version of quick-sort. Perform benchmarking tests to determine the range of values of n where quick-sort is on average better than insertion-sort.
- P-12.64 Design and implement a version of the bucket-sort algorithm for sorting a list of n entries with integer keys taken from the range $[0, N - 1]$, for $N \geq 2$. The algorithm should run in $O(n + N)$ time.
- P-12.65 Implement an animation of one of the sorting algorithms described in this chapter, illustrating key properties of the algorithm in an intuitive manner.
- P-12.66 Design and implement two versions of the bucket-sort algorithm in Java, one for sorting an array of **byte** values and one for sorting an array of **short** values. Experimentally compare the performance of your implementations with that of the method, `java.util.Arrays.sort`.

Chapter Notes

Knuth’s classic text on *Sorting and Searching* [61] contains an extensive history of the sorting problem and algorithms for solving it. Huang and Langston [49] show how to merge two sorted lists in-place in linear time. The standard quick-sort algorithm is due to Hoare [45]. Several optimizations for quick-sort are described by Bentley and McIlroy [15]. More information about randomized algorithms can be found in the book by Motwani and Raghavan [75]. The quick-sort analysis given in this chapter is a combination of the analysis given in an earlier Java edition of this book and the analysis of Kleinberg and Tardos [57]. Exercise C-12.33 is due to Littman. Gonnet and Baeza-Yates [38] analyze and compare experimentally several sorting algorithms. The term “prune-and-search” comes originally from the computational geometry literature (such as in the work of Clarkson [22] and Megiddo [70]). The term “decrease-and-conquer” is from Levitin [66].